



Less is More: Feature Selection for Autonomous Navigation

Kevina Ma
UC Berkeley

Background/Introduction

In FIRST Tech Challenge (FTC) robotics, teams face limited time, budget, and few matches on unfamiliar fields during competitions. They need to decide what additional features to add to their robot to help it perform the best during the pre-programmed autonomous period, regardless of what the field looks like. How can they find the best feature given performance and cost? It can be difficult for teams because there are 2 main issues that can throw off an autonomous program: pose error and obstacle error. These two failure modes aren't specific to FTC. They affect any robot navigating from an internal map, such as a self-driving car misjudging its lane. Simulation is a tool that enables teams to test features cheaply before investing in physical hardware.

Approach

To find the best feature for the cost, every possible sensor combination was built as a virtual robot. They were each ran through thousands of simulated matches under a range of realistic field conditions. For each build, how often the build succeeded and what that build cost were recorded.

Pillar 1: Simulate the Field and Robot Behavior

- A virtual version of the FTC field was built and tested on 3 different layouts, with the start and goal randomized each run.
- The robot plans its route using A*, a pathfinding algorithm chosen after being compared against 4 others.
- While the robot drives that planned route, the type of things that go wrong during competitions are added in, such as its sense of its own position slowly drifting off, or the field not matching what it expected.

Two independent ways an autonomous run fails

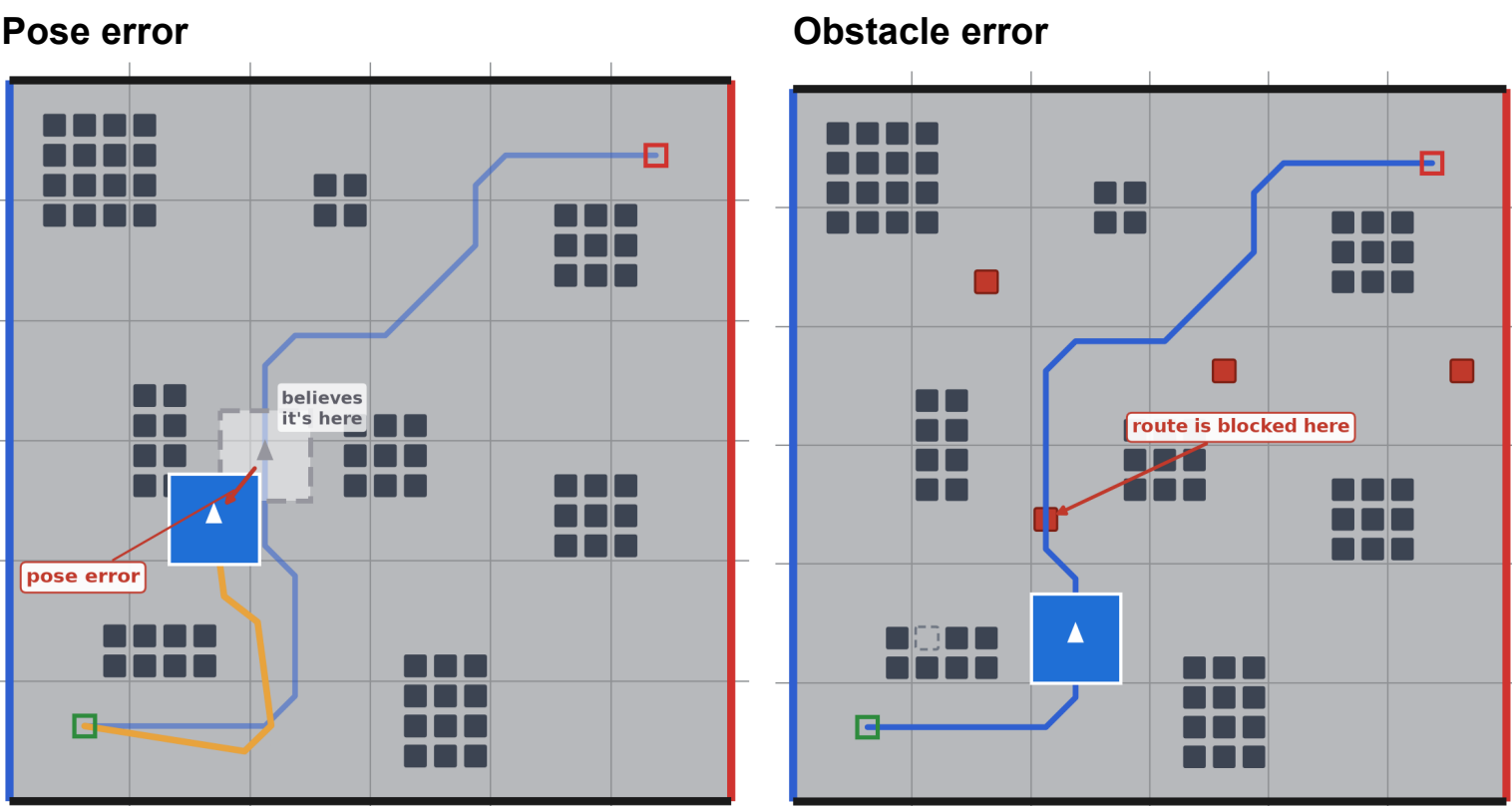


Figure 1. Left (pose error): the robot's belief diverges from its true position. Right (obstacle error): the actual field doesn't line up with the map. Simulated independently to isolate each effect.

5 pathfinding algorithms tested

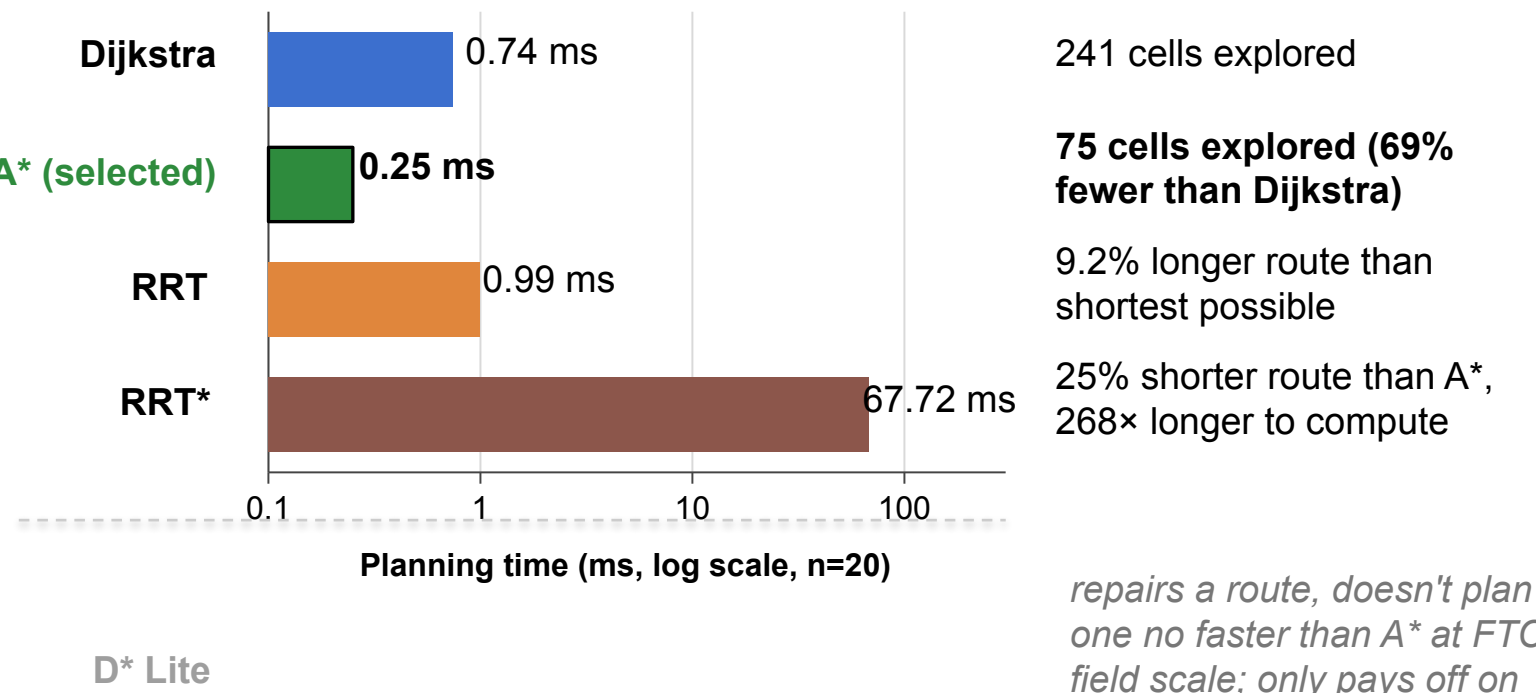
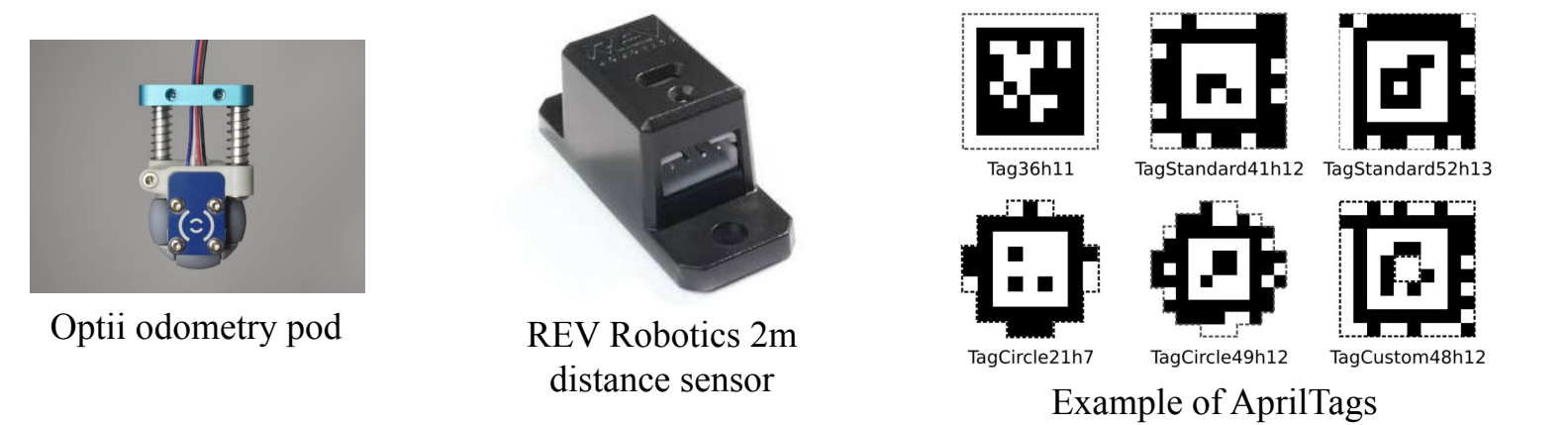


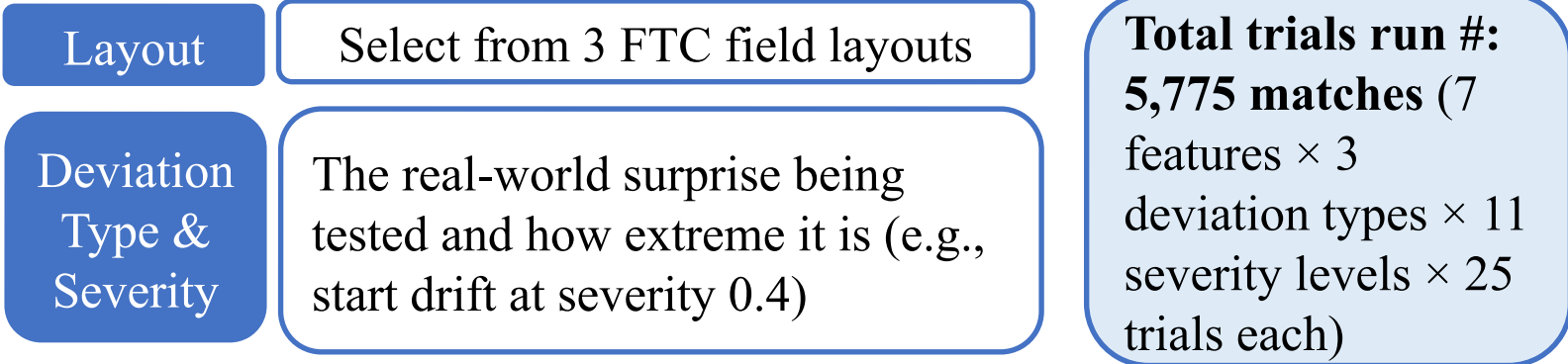
Figure 2. Planning time for 5 pathfinding algorithms on the simulated field; A* matched Dijkstra's path exploring 69% fewer cells.

Pillar 2: Select Features to Test

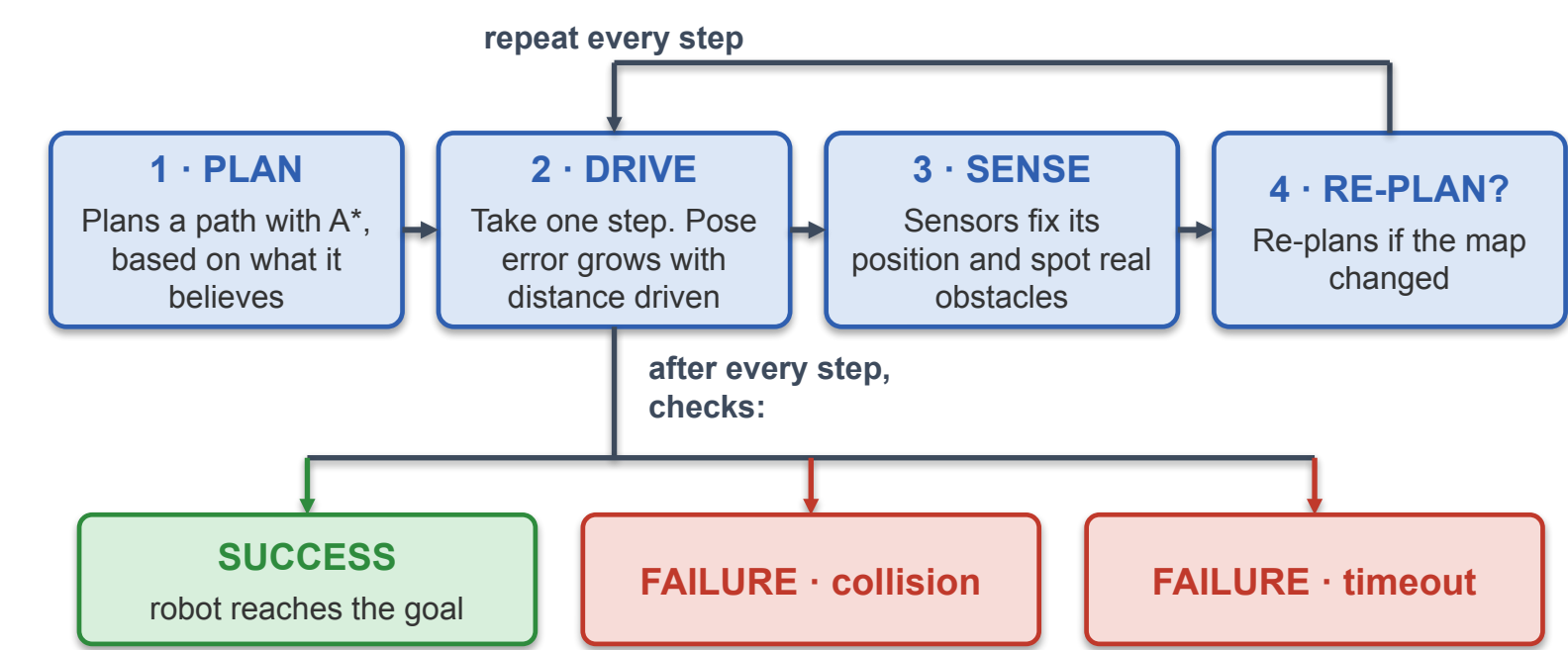
Features & Cost	Functionality
A. No-sensor baseline - \$0	Estimates position by counting wheel rotations alone
B. Odometry pods (3x) - \$195	Tracks true motion with unpowered, non-slip wheels
C. Distance sensors (3x) - \$95	Measures the distance to nearby objects
D. AprilTag front camera (standard USB webcam) - \$25	Reads printed markers on the field walls to determine location
E. Second rear camera - \$25	Reads the same markers from behind the robot, doubling camera coverage
F. IMU (Inertial Measurement Unit) - \$0	Senses the robot's rotational position
G. Mecanum wheels - \$170	Lets the robot slide sideways without turning first



Pillar 3: Testing/Trial Phase



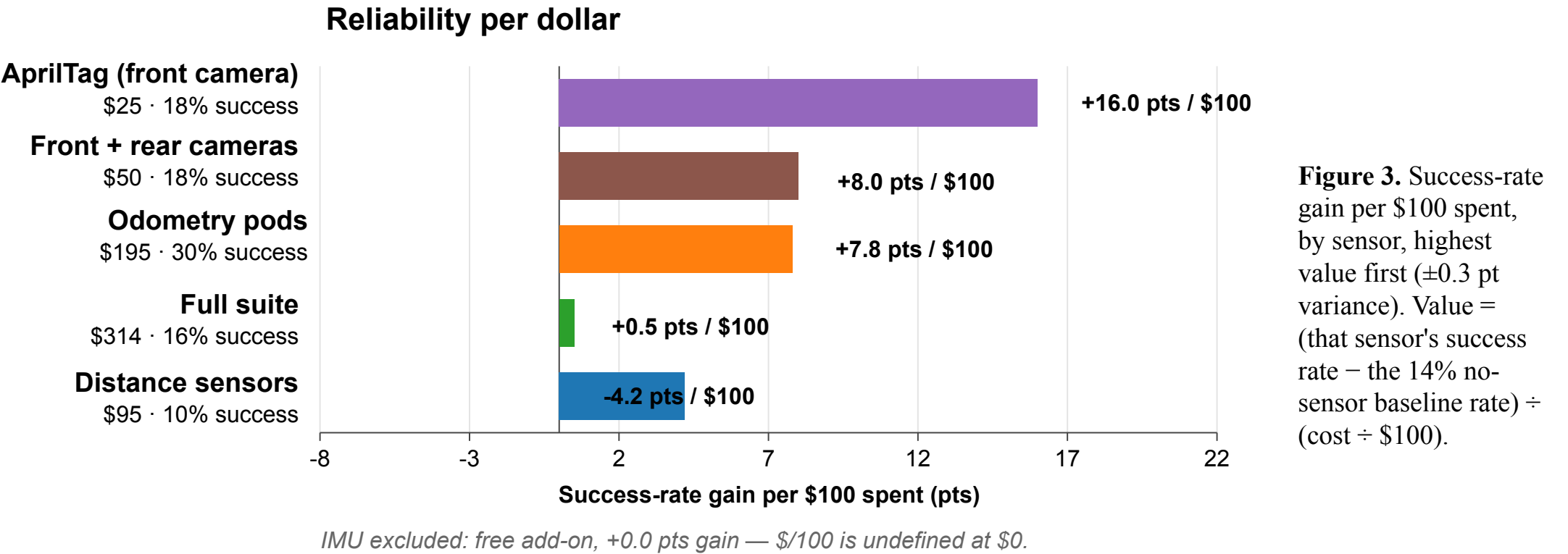
How one simulated match works



Results

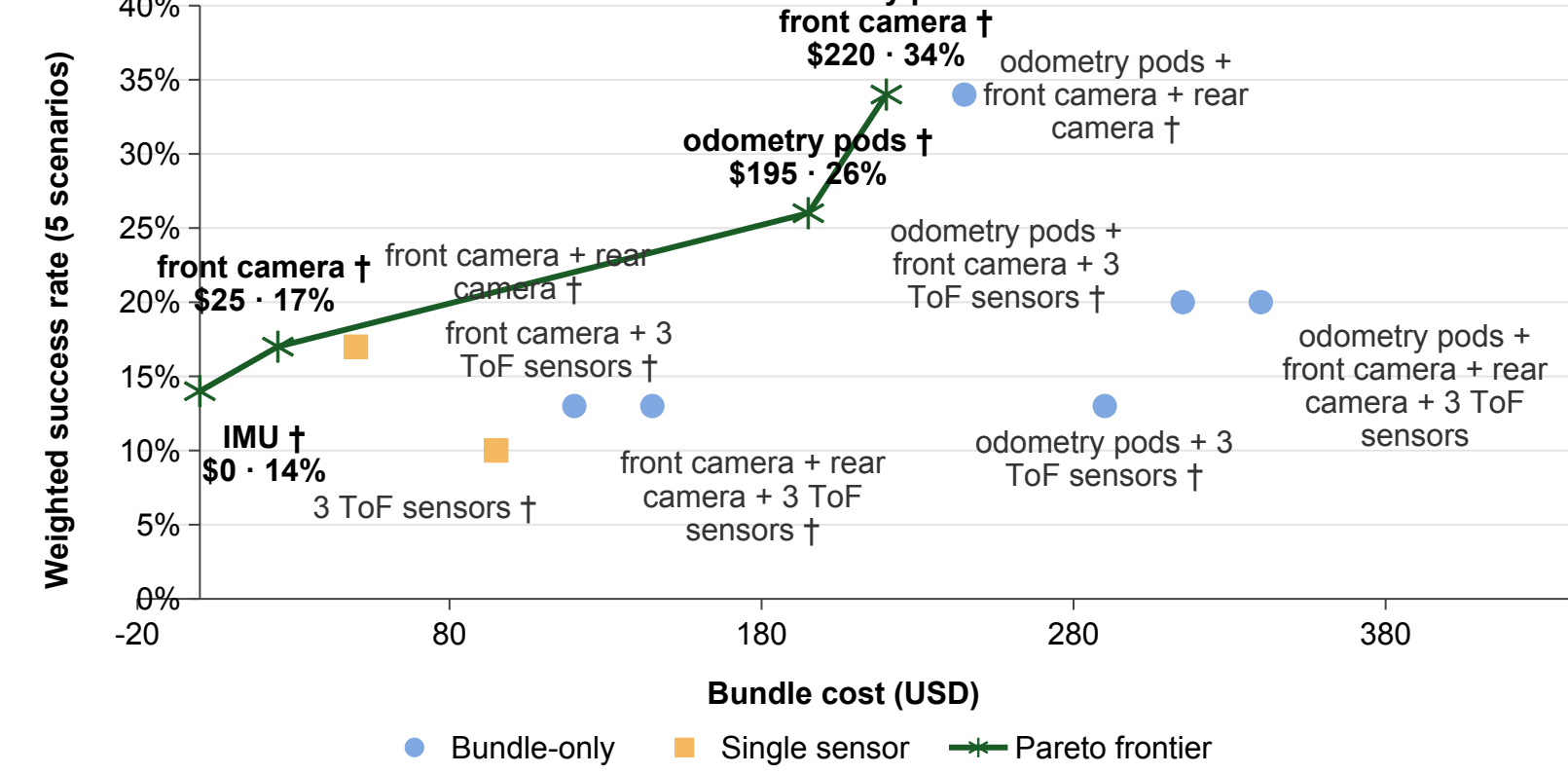
Which single sensor gives the best performance for its price?

- The \$25 AprilTag front camera is the best-value upgrade at 18% success rate. Odometry pods have the best raw performance (30% success) at \$195 and are about tied with the cheaper front + rear camera pair on value. The \$314 full suite is the worst value tested.
- Distance sensors and the mecanum-wheel upgrade leave the robot worse off than not buying them at all.



Which combination of sensors is worth the extra cost?

- The best combination tested is odometry pods + AprilTag front camera (\$220 total). Pairing a pose fix with a pose-correction fix surpasses every other buildable robot, including pricier ones.
- Every combination that includes distance performs underperforms the same bundle without them.
- Once the failure modes are covered, more parts are not necessary and can cause underperformance.



† IMU (free) never changes cost or success -- "+IMU" bundles overlap their twin exactly.

Figure 4. Every buildable sensor combination, plotted by cost vs. success rate. The frontier marks the smart buys.

Which build is best if you know your likely failure mode?

- The \$245 bundle (odometry pods + front + rear camera) is the best overall pick as it is the best or near-best choice in 4 of 5 scenarios tested.
- If the failure is a tight, corridor-heavy match, the rear camera can be dropped because it doesn't add anything to the performance.
- Heavy pose drift stays the hardest failure mode no matter what you buy and is a sign of a hardware/calibration gap.

Which robot wins a scenario?

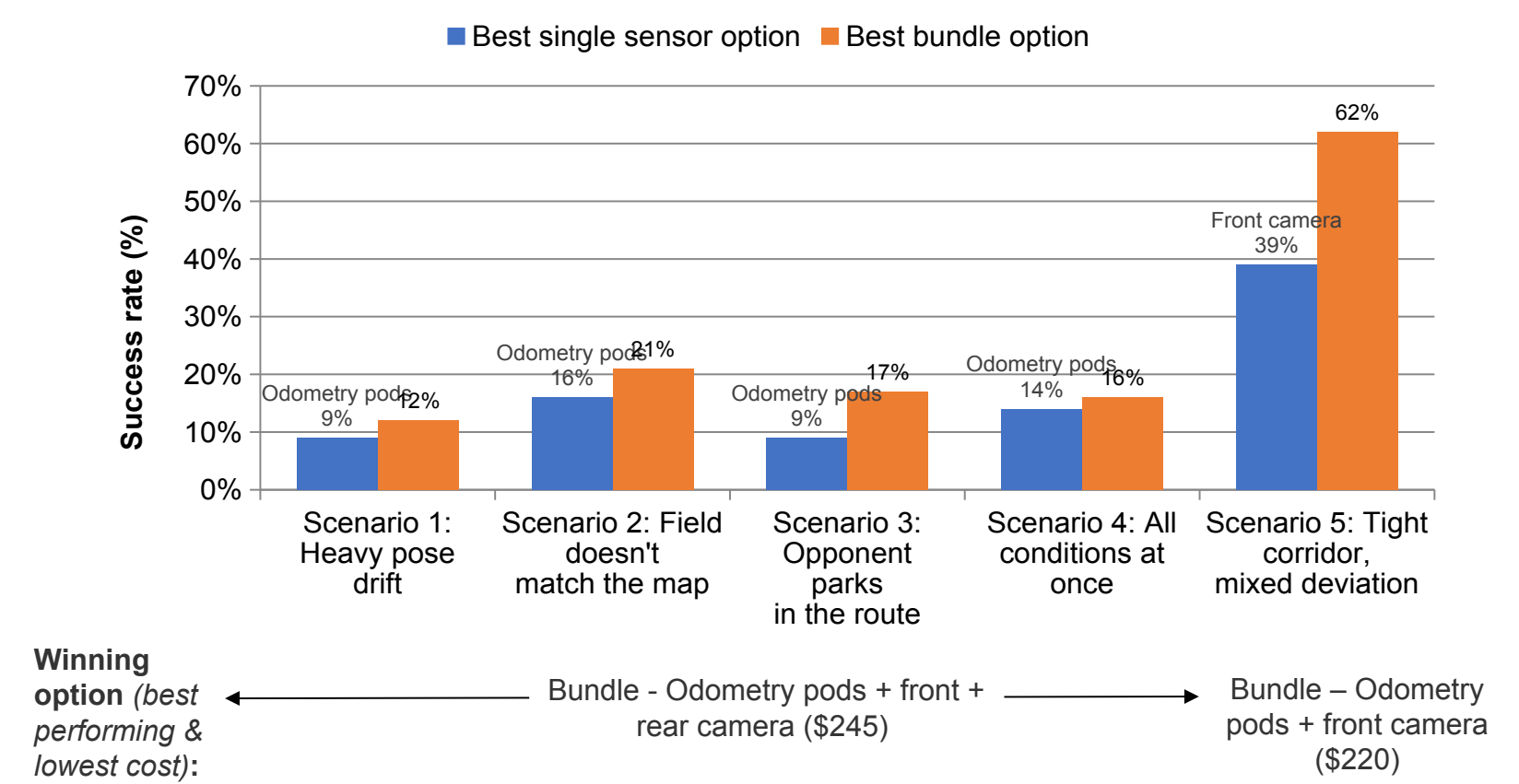


Figure 5. Scenario-best pick (blue = single sensor, orange = bundle), across 5 scenario types.

Conclusion

Spending more didn't make the robot more reliable. A feature was only worth buying if it fixed a problem the robot actually had, and stacking sensors that covered the same failure wasted money without reducing risk. There's no universal best build. The right combination depends on what a team expects to go wrong on the field.

Next Steps

- Test with real FTC field and physical robot
- Finetune layout and deviation model against a real, surveyed field
- Generalize framework to other domains with the same pose/obstacle-error failure mode, e.g. home robots, warehouse AMRs, search-and-rescue robots in unmapped or hazardous terrain

Acknowledgements

Image credits: AprilTag example, FIRST Tech Challenge Docs; odometry pod photo, Optii; distance sensor photo, REV Robotics.

Independent project, inspired by my time on an FTC team in high school.